Применение ИИ-ассистента для разработки unit-тестов DB Monitoring

Домашнее задание B15 «ИИ как соавтор кода: prompt engineering», слайд 41; Issue #44. Выбран вариант №5 практики (слайд 31): 3–5 unit-тестов на pytest для существующей функции, включая крайние случаи. Дата работы: 14.11.2026, база — `master 3fb9d2c`.

1. Задача и ассистент

Ассистент: Claude Code (CLI-агент), модель Claude Opus 5.5. Он читал код, запускал команды и писал файлы в локальной копии репозитория; результат проверяет человек.

Функция: `ml.features.validate_vector` — проверка четырёх именованных признаков (`row_count`, `null_rate`, `d_row_count`, `d_null_rate`) перед применением сохранённого scaler. Её вызывают сервис инференса, реестр моделей и мониторинг модели. Задача средней сложности: в функции есть проверка состава, типов, физических границ, восстановление предыдущего наблюдения (`текущее − дельта`) и допуск `1e-12` на ошибку округления.

Уже проверено косвенно: типы и `True`/`None`/строка, NaN, ∞, переполнение (`tests/test_model_registry.py`); отрицательный `row_count` и невозможная `d_row_count` через HTTP (`tests/test_inference_service.py`); границы диапазонов в JSON-схеме контракта (`tests/test_openapi_contract.py`) — это проверка схемы, а не самой функции. **Не проверялись:** точные значения на границах, невозможный предыдущий `null_rate`, допуск на округление, состав набора признаков и порядок результата.

2. Промпт

Использован один промпт пользователя: полная инструкция по Issue #44 (цель, изучение проекта, ограничения, проверки, отчёт). Ключевая часть — рабочее задание, дословно:

> «Выбери одну существующую функцию DB Monitoring с недостаточно покрытыми крайними случаями. Подготовь для неё 3–5 осмысленных unit-тестов на pytest. Не дублируй существующие тесты и не изменяй production-код. Проверь выполнение тестов и зафиксируй фактические результаты».

Там же требования, дословно: «Используй существующие соглашения проекта по pytest», «Не подгоняй тесты под ошибочное поведение без его анализа», «Не меняй production-код ради прохождения тестов». Уточняющих промптов не потребовалось: задание выполнено по этому одному сообщению.

3. Что сделал ассистент

1. Просмотрел функции в `app/` и `ml/`. Нашёл все вызовы `validate_vector`, `build_vector` и `build_rows` в коде и тестах и выбрал `validate_vector`, потому что её граничные случаи напрямую не проверялись.
2. До написания тестов запустил функцию на граничных входах и зафиксировал её фактическое поведение: значения 0 и 1 принимаются, предыдущий `null_rate` −0,1 и 1,1 отклоняется, `0.3 − (0.1 + 0.2)` = $-5{,}55 \cdot 10^{-17}$ укладывается в допуск. Дефектов не обнаружено.
3. Написал `tests/test_features.py`: 5 тестов, 10 случаев с параметризацией. Стиль проекта: функции pytest, `parametrize` с `id`, проверка текста ошибки через `match`.

Тест	Что проверяет
`test_exact_physical_boundaries_are_accepted`	Пустая таблица (все признаки 0) и переходы `null_rate` 0→1 и 1→0 принимаются; результат — числа `float`
`test_previous_null_rate_outside_unit_interval_is_rejected`	`null_rate − d_null_rate` < 0 или > 1 — ошибка «impossible previous observation»
`test_subtraction_roundoff_at_zero_previous_null_rate_is_tolerated`	Ошибка округления в пределах `1e-12` допускается, отклонение на `1e-9` — уже нет
`test_feature_set_must_match_names_exactly`	Отсутствующий и лишний признак, позиционный список вместо словаря — ошибка
`test_vector_order_follows_feature_names_not_input_order`	Порядок результата задаёт `FEATURE_NAMES`, а не порядок ключей входа

4. Недостатки и корректировки

- **Гипотеза для теста допуска не подтвердилась.** Сначала ассистент искал случай округления, который даёт сама `build_vector`. Перебор 200 000 случайных пар и сетки значений показал: её арифметика не выводит предыдущий `null_rate` за [0, 1]. Поэтому в тесте использована дельта, посчитанная иначе (`0.1 + 0.2`), — так бывает у внешнего клиента. Реалистичного примера для верхней границы (1) не нашлось, поэтому проверяется только нижняя.
- **Версии инструментов.** Во временное окружение сначала установились последние pytest 9.2.0 и ruff 0.17.5. До запуска тестов они заменены на версии проекта из `requirements-dev.txt`: 8.3.4 и 0.9.4.
- **Код тестов не исправлялся:** первый запуск pytest и ruff прошёл без ошибок. Production-код не изменялся.

5. Результаты проверок

Проверка	Фактический результат
`pytest tests/test_features.py -v`	10 passed за 0,06 с (pytest 8.3.4, Python 3.12.3)
`ruff check` / `ruff format --check`	All checks passed / 1 file already formatted
Ловят ли тесты ошибки	7 намеренных искажений функции во временной копии: без проверки предыдущего `null_rate`, без допуска, слишком широкий допуск, лишние ключи разрешены, строгие границы, порядок входа, без приведения к `float`. Каждое роняет 1–3 случая из 10

Проверка	Фактический результат
Дублирование	Имена тестов уникальны; сценарии сверены с тремя файлами из раздела 1
`git diff --check`, состав изменений	Без замечаний; изменены только тест, этот отчёт и его PDF

Не запускались: полный набор тестов и CI. Остальные тесты проекта требуют зависимостей ML и сервиса, которые для этой задачи не устанавливались.

6. Выводы

Польза. Ассистент быстро собрал по трём тестовым файлам, что уже проверено, и предложил сценарии без дублей. Полезнее всего оказалось правило «сначала запустить функцию, потом писать ожидания»: тесты описывают фактическое поведение, а не догадку. Искажение функции быстро показывает, проверяют ли тесты что-нибудь на самом деле.

Ограничения. Первое предположение ассистента о допуске оказалось неверным и обнаружилось только при запуске кода. Без явного указания он берёт последние версии инструментов вместо версий проекта. Отчёт о процессе составлен тем же ассистентом, поэтому промпт, тесты и результаты проверок нужно сверить человеку. Чёткий промпт с запретами («не дублируй», «не меняй production-код», «не подгоняй тесты») заметно сузил пространство решений, и уточнения не понадобились.